



Chapter 1

Introduction

In 1988, a fascinating little piece of code hits the limelight. That year, one of the winners of the annual International Obfuscated C Code Contest features a program that writes out to the terminal console the text of an eighteenth-century poem titled *The Twelve Days of Christmas*. Figure 1.1 shows the first three verses of the text, as they appear on the output console when executing the code. This poem is particular in that its text has a regular structure. Text with such a structure is amenable to being constructed by software in a way that goes beyond printing hard-coded data. With a poem like *The Twelve Days of Christmas*, there was thus opportunity for creating a clear and compact solution for displaying a poem on the console. However, as promised by the name of the contest where it was featured, the program is anything but clear. In fact, its inner workings are unfathomable. Figure 1.2 reproduces the complete code of the program.

```
On the first day of Christmas my true love gave to me
a partridge in a pear tree.

On the second day of Christmas my true love gave to me
two turtle doves
and a partridge in a pear tree.

On the third day of Christmas my true love gave to me
three French hens, two turtle doves
and a partridge in a pear tree.
...
```

Fig. 1.1 Partial output of *The Twelve Days of Christmas* program of Figure 1.2

This quirky piece of computer science trivia illustrates the impact of a lack of self-evident structure in software. Here, we have a programming problem with trivial requirements: the functionality of interest requires no input and produces a sin-

```

main(t,_,a ) char* a;{return!0<t?t<3?main(-79,-13,a+main(-87,
1-,main(-86, 0,a+1 )+a)):1,t<_?main( t+1, _, a ):3,main(-94,
-27+t, a )&&t == 2 ?_<13 ? main ( 2, _+1,"%s %d %d\n" ):9:16:
t<0?t<-72?main( _, t,"@n'+,#'/*{}w+/w#cdnr/+,{}r/*de}+,/*{*+\\
,/w{%+,/w#q#n+,/#{1,+,/n{n+,/+#n+,/#;#q#n+,/+k#;+*,/'r : 'd*'\\
3,}{w+K w'K:'+}e#';dq# 'l q#'+d'K#!/+k#;q# 'r}eKK#}w' r}eKK{n l}\\
' /#;#q#n' ) { }#w' ) { }{n l}' /+ #n' ;d}rw' i;# ) {n l}! /n {n#'; r {#w' r\\
nc{n l}' /# {l,+'K {rw' iK{;[{n l}' /w#q#n' wk nw' iwk{KK{n l}! /w {\\
% 'l# #w# ' i; : {n l}' /* {q# 'ld; r' } {n lwb! / *de} 'c ; ; {n l' - { }rw}' / +, \\
}## ' * }#nc, { , #nw}' /+kd' +e}+; # 'rdq#w! nr' / ' ) }+} {rl# ' {n' ' }# \\
}'+}## (!! /") :t<-50?_==*a?putchar(31[a]):main(-65,_,a+1):main
(( *a == '/' )+t, _,a+1):0<t?main ( 2, 2 , "%s") : *a=='/' ||main
(0,main(-61,*a,"!ek;dc i@bK' (q)-[w]*%n+r3#l,{ }:\\nuwloca-0;m\\
.vpbks,fxntdCeghiry"),a+1);}

```

Fig. 1.2 Source code of the 1988 *The Twelve Days of Christmas* C program by Ian Phillips. This code compiles and executing it will produce the output illustrated in Figure 1.1. © 1988, Landon Curt Noll and Larry Bassel. Reproduced with permission.

gle, unchangeable output. Yet, the code to support this functionality cannot be understood by a normal human being. But what is the problem, if the code works?

Software needs to change, and for software to change, at least one person must be involved at some point. Software needs to change for a variety of reasons, from fixing bugs to adapting the code to an evolving world. For example, many of the gifts referred to in the poem are European birds (e.g., partridge, turtle doves, French hens). Contemporary software development best practices include the *localization* of software applications, namely, the option to tailor a software application to account for region-specific characteristics. It would thus be nice to adapt the code of the application to replace the name of European birds to some that readers could relate to based on their own region (for example, to replace *partridge* with *turkey* for North American users). To modify a piece of code, however, one must understand its structure, and this structure must, to a certain extent, accommodate the change. In the case of *The Twelve Days of Christmas*, any ambition to ever change the code is hopeless.

The example of *The Twelve Days of Christmas* is facetious for the sake of illustration. Because this code was obfuscated on purpose, it would be comforting if we could discount it as irrelevant. Unfortunately, because writing messy code is often the path of least resistance in the complex social, technological, and economic reality of software development, badly designed code is not hard to find. For example, in a famous high-profile case where automotive software was determined by the courts to be responsible for a fatal accident, the experts who reviewed the software likened its structure to that of a bowl of spaghetti. Whether code is cryptic purposefully or accidentally, the result is similar: it is hard to understand and change without introducing errors.

To explore the contrast, let us design a version of the program where the structure is evident. Consistently with the rest of the code in this book, the program is in Java. First, we tackle the issue of producing the first line of a verse:

```
static String[] DAYS = {"first", "second", ..., "twelfth"};

static String firstLine(int day) {
    return "On the " + DAYS[day] +
        " day of Christmas my true love gave to me:\n";
}
```

This code is clear because the function is short, it abstracts an obvious concept (the creation of the first line), and the only parameterization involved maps directly to the problem domain (changing the day).

The second sub-problem is to create the list of gifts for a given day. In this case we can leverage the inherent recursion in the poem's structure to organize the code in a function that creates a list of gifts by adding the last gift to a smaller list of gifts:

```
static String[] GIFTS = { "a partridge in a pear tree",
                          "two turtle doves", ... };

static String allGifts(int day) {
    if (day == 0) {
        return "and " + GIFTS[0];
    }
    else {
        return GIFTS[day] + "\n" + allGifts(day-1);
    }
}
```

The `allGifts` function provides a classic implementation of a recursive algorithm. In this case, the code's structure is explicit because it directly realizes a foundational strategy in computing.

At this point the only thing left is to put the poem together by assembling the twelve verses. Here, the only small issue is that, in the first verse, we do not add the word *and* in front of *a partridge*. No matter how small a program, it can be difficult to completely avoid annoying corner cases.

```
static String poem() {
    String poem = firstLine(0) + GIFTS[0] + "\n\n";
    for (int day = 1; day < 12; day++) {
        poem += firstLine(day) + allGifts(day) + "\n\n";
    }
    return poem;
}
```

At a glance, we see the overall structure of the code: a special case for the first verse, then an iteration through the remaining eleven verses, where each verse is created by concatenating the output of two functions: one to create the first line, and the other to create the list of gifts.

1.1 Defining Software Design

Software design is a mysterious activity. For many software development projects, “the design” is not necessarily something one can retrieve and look at. Similarly, very few people walk around with the title of “software designer”. In that sense, designing software is not like designing furniture or clothing.

The word *design* is both a verb and a noun, so it can refer to both a process (*to design*) and the outcome of this process (*a design*). My working definition of software design (the process) is *the construction of abstractions of data and computation and the organization of these abstractions into a working software application*. At first this may sound overly restrictive, but when we consider everything that the term *abstraction* can mean (variables, classes, objects, etc.), we see that we are afforded quite a bit of flexibility for interpreting what software design means.

In practice, the design process is essentially one of decision making. *Should we use a list or a stack? What services should this interface offer? Where should this error be handled?* Considering design as decision making leads to the concept of a *design space*. A design space can be imagined as an n -dimensional geometric space where each dimension corresponds to a design quality attribute. Typical design quality attributes for software include understandability, reusability, and ease of implementation. Within such a design space, each specific design decision (or coherent set of decisions) corresponds to a coordinate in the space that represents the consequence of the decision. Figure 1.3 illustrates the idea with two dimensions. In practice, any design decision is likely to be good in some dimension, but less good in other dimensions, something we call a *design trade-off*.

Two partitions of the design space that are useful to consider are the space of *possible solutions*, and the space of *acceptable solutions*. We can observe that the ideal solution, which is optimal in all dimensions, is unlikely to be possible. In other words, given a design problem, there is not necessarily a single solution that is the “right answer”, only solutions that are better or worse in some dimensions (but including some solutions that are pretty bad in most dimensions).

The concept of a design space may make it look like selecting a design decision is a systematic process. This is not the case. Where the analogy breaks down is that a geometric space is completely defined, whereas the reality of software design is rife with uncertainty. First, not all possible decisions are known and, in complex situations, there may be an infinity of them. Second, estimating to what extent a design decision fulfills a given quality attribute (e.g., understandability) is an approximate process. Consequently, there is no standard formula for arriving at a point in the design space. In most realistic software development contexts, it will not be the case that to design and implement a software requirement, we can follow a pre-determined set of steps. Software design is a *heuristic* process: it consists of iterative problem-solving guided by experience and know-how (see Section 1.4). In fact, the heuristic nature of the software design process is what makes it an exciting creative activity.

The quality attributes that constitute the dimensions of the design space also correspond to the general goals of design. One of the most important goals for soft-

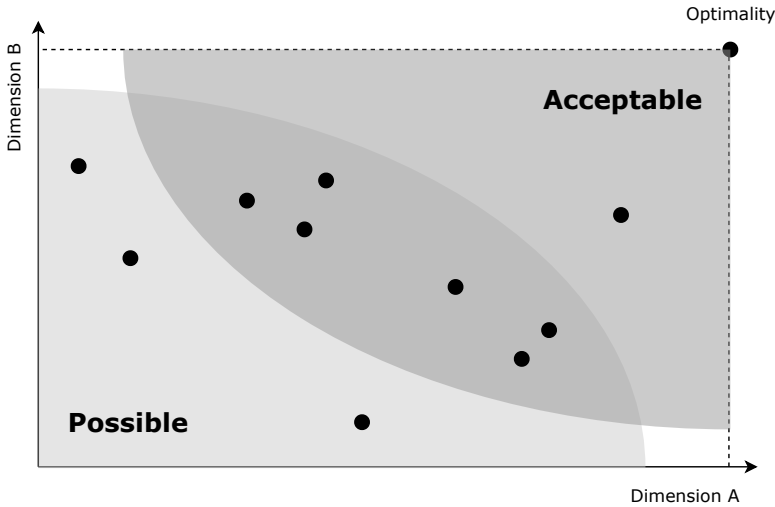


Fig. 1.3 A hypothetical design space

ware design is to reduce the complexity of software, which means making it easier to understand. Cleanly-designed code that is easy to understand is less error-prone and easier to modify. In contrast, messy code obscures the important decisions of its original developers. When developers ignore existing design constraints, they risk modifying code in a way that does not agree with the original structure, and thereby introduce errors and generally degrade the quality of the code. The problem of modifying code in a way that does not respect the original structure has been called *ignorant surgery*.

In general, the relative importance of design goals depends on the context in which a piece of software is being designed. A *design context* (or problem) is a specific set of requirements and constraints within a domain in which a design solution must be found and integrated. For example, because of economic or contractual reasons, it may be required to design a particular piece of software to maximize its reusability. Or, if a piece of software is intended to be integrated into safety-critical applications, it may be more important to prioritize robustness (i.e., resilience to errors). In this book, I give a lot of importance to the *understandability* quality attribute. I try to emphasize designs where the code itself reveals the underlying design decisions and the *intent* behind these design decisions. The idea of having design decisions be self-evident in code is a property I call *sustainability*.

If we consider that the design process is a series of decisions-making activities about software abstractions, then it follows that a good definition for a *design* is a cohesive collection of these decisions. This definition for a design artifact is sufficiently general to avoid dictating the medium in which the design is captured. In formal software development settings, this could be an official standardized design

document. In less formal contexts, design decisions could be stored in the code, diagrams, or various documentation pages associated with the project. In the extreme, design decisions could exist only in the mind of the developers who made them. Because people tend to forget or misremember, this latter approach is best kept to a minimum. Section 1.3 provides an overview of how design knowledge can be captured.

1.2 Design in the Software Development Process

Design is only one of the many activities that take place during the development of a software system. There is an abundant literature on different process models for software development. A *process model* describes (and sometimes prescribes) how the different steps required to create a system are organized. Different process models offer different ways of doing things for different reasons. In the early days of the software engineering discipline it was believed that a *planning-heavy* process, exemplified by the waterfall software process model, was the desirable way to build high-quality software. However, in the mid-1990s this belief was challenged by a movement towards a more organic approach to software development, also called *agile development*. In practice, ideas about how to best develop software keep evolving, and in the end the important things are to have a development process in the first place, and for that process to be well-adapted to the type of system being developed and the organization that develops it. For example, the process used by an organization to develop a prototype for a video game would probably be different from the process used to develop banking or aeronautical software.

The issue of devising, adapting, or even following a software development process is not the main focus of this book. However, even when learning about software design, it is useful to have a general idea of software development processes, if only to stay oriented in the wide and buzzword-laden realm of technology.

One concept of the software development process literature that is related to software design is the idea of a software development *practice*. A practice is a well-understood way of doing something to achieve a certain benefit. An example of a practice many programmers are familiar with is *version control* (the use of software tools to keep track of changes to software development artifacts). Another example of software development practice is *pair programming* (writing code as a team of two in front of a single computer). In this book I refer to a number of software development practices that directly support good design, including the use of *coding conventions* (see Appendix B) and *refactoring* (see below).

Another concept of software development processes that is relevant to software design is that of the *iteration*. As discussed in Section 1.1, when searching for a design solution, it is usual to iterate over various alternatives. However, iterations also take place at a more macroscopic level in software development, in the sense that the design of the system may be periodically extended, reviewed, and/or improved. In some cases, the design can even be improved without any change to the

observable behavior of the system. Improving the design of code without changing its functionality is the software development practice known as *refactoring*. There are various reasons why refactoring can become necessary or desirable. One reason is that the original developer did not really get it right, and after working with the code for a while it becomes apparent that a different design would be better. Another reason is that we might want to add modules and features that do not integrate well with the existing design, so we first refactor the design to prepare it so that it better supports the later addition of new code. A third reason is to reduce accumulated design weaknesses. As part of maintaining the code (e.g., to fix bugs), developers occasionally implement quick and dirty solutions that do not align properly with the existing design. This phenomenon is known as accumulating *technical debt*. By not investing the effort necessary to code a clean solution, the team effectively borrows development effort from the future. If allowed to accumulate, too much technical debt can threaten the viability of the project, just like the risk of bankruptcy incurred by excessive borrowing in the financial sense. When technical debt is incurred in a project, refactoring is a way to pay it back, and good software development teams will periodically refactor their code. Thus, software design is in continual evolution.

1.3 Capturing Design Knowledge

A design (or design solution) is a collection of decisions, each of which is the result of a search process through a design space for a particular design problem, or context. In practice, a design decision is a statement about how to organize abstractions to meet a requirement, ideally associated with the reason for this statement. A simple example could be: *We will store the appointments in a list because we need to know in what order they were added.* For this decision to even exist, it has to be in at least one developer's mind at some point. We thus have a first medium for storing design decisions: a person's mind. For small projects, this could be sufficient. However, given that human memory is unreliable, it can be worthwhile to record important design decisions externally. This raises the question of how to capture design knowledge. The following is a concise summary of the options for externalizing design knowledge:

- **Source code:** Many design decisions can be captured directly in the source code. The example above, of selecting a list as a data structure, would be one case. The advantage of source code is that it is a formal language whose rules are checked by the compiler. Unfortunately, source code is not a good substrate for capturing the *rationale* for design decisions. For this purpose, code comments can be of some assistance.
- **Design documents and diagrams:** Design decisions can be captured in documents specifically aimed at capturing such design decisions. There exists a wide variety of formats for documents about software, from standardized design doc-

uments to blog posts. Design documents may also include diagrams, which are another way to represent design decisions.

- **Communication and collaboration tools:** Design information can be captured in email and comments stored in tools used for software development, such as issue management systems and version control systems.
- **Specialized models:** In certain software development projects, developers use formal models to specify various aspects of the software. These models can then be automatically converted into code in a programming language. Such an approach is called *generative programming* or *model-driven development* (MDD). In model-driven development, the models serve as design documents. As a software construction approach, model-driven design and development is outside the scope of this book.

Because the level of design abstraction covered by this book remains close to the source code, many of the design decisions discussed will be at least partly reflected in the code. Subsequent chapters will also contain many diagrams and accompanying text that document design decisions.

The Unified Modeling Language

There will often be situations where we need to discuss design problems and solutions that are impractical to describe using either source code or natural language. For this purpose we can use a specialized *modeling language*. This situation is not limited to software. For example, describing instrumental music in plain language is near-impossible: instead, we use musical notation.

Historically, many different modeling languages and notations have been developed for representing, at an abstract level, various aspects of a software system. This disparity was, however, an obstacle to adoption because of the overhead involved in interpreting models expressed in an unfamiliar notation. Thankfully, in the mid-1990s the main software modeling notations were merged into a single one, the Unified Modeling Language (UML), which was subsequently adopted as a standard by the International Organization for Standardization (ISO).

The UML is a modeling language organized in terms of different types of diagrams intended to illustrate different aspects of software. Examples of design information that can be neatly captured in the UML include relationships between classes (e.g., `A` inherits from `B`), changes in the state of an object (e.g., the `list` object goes from *Empty* to *Non-Empty* when the first element is added), and sequences of calls dispatched on objects (e.g., `a.m1()` results in a call to `b.m2()`).

Not all development teams use the UML. However, those who do can use it in different ways for different reasons. For example, UML can be used to produce formal design documentation in waterfall-type development processes. Others use the UML to describe enough of the software to be able to automatically generate the code from the models, following the idea of generative programming. In this book, I use the UML simply for *sketching* design ideas. The diagrams included in this book are not expected to be automatically transformable into code. I also use

the smallest subset of the modeling language necessary, and introduce the notation progressively.

An important thing to remember about UML diagrams is that they are *models*. This means that they are not intended to capture every single detail of a solution. Ideally, a UML diagram will focus on illustrating a *single main idea* and only include the relevant information. In UML diagramming it is a common practice to leave out the parts of a system and details that are not directly relevant to the ideas or design decisions being represented.

1.4 Sharing Design Know-How

Capturing knowledge about the design of a particular system is one thing, but how do we capture general knowledge about how to design software? Software design is influenced by the skills and experience of the designer, and this type of heuristic knowledge is not easy to synthesize, package, and share. In earlier days, organized approaches to share design know-how centered around structured *analysis and design methods*, which prescribed a sequence of steps and the use of specialized charts and other instruments. Such approaches peaked in the 1980s, and were replaced with adaptations suited to object-oriented programming, a paradigm that was then quickly gaining adoption. Comprehensive object-oriented design methods themselves peaked in the mid-1990s. In the meantime, a number of *design principles* were becoming increasingly recognized and accepted, and it was being observed that some elements of design solutions tended to recur between many applications, a concept that would become known as *design patterns* and *antipatterns*.

Software Design Principles

Design principles are general ideas that guide decision making when searching for a solution, without being strict rules that can be followed systematically. For example, *Loose Coupling* is a commonly referenced principle of software design that states that different parts of a software (such as classes) should have as few dependencies between them as possible. Another example is *Separation of Concerns*, which suggests to organize software entities such that each one targets a single area of focus, or “concern”. There is no official list or taxonomy of software design principles, and different principles can be described or interpreted differently by different people. Because design principles are abstract ideas, they require practice and experience to apply properly. In this book, I cover specific design principles in detail where they are most relevant in terms of the overall organization of the material.

Design Patterns

The idea of reusing elements of object-oriented design was captured in the concept of a *design pattern* in the book *Design Patterns: Elements of Reusable Object-Oriented Software* [7]. This book, often referred to as the *Gang of Four* book from the author list, is one of the most influential software design books in existence. Following the concept of an architectural pattern originally proposed by an architect named Christopher Alexander, the book describes 23 patterns for addressing common software design problems. Since then, countless other patterns have been documented. The idea to capture abstract design solutions that address specific problems was a breakthrough for software engineering, because it provided a practical way to convey design know-how and experience without the requirement to adopt a comprehensive design method. To this day, design patterns and close variants of the concept have been a dominant way to capture design know-how. There currently exist countless design catalogs for different programming languages, in the form of books and websites.

According to the Gang of Four, a pattern has four essential elements:

The pattern name [...] Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction [...] The *problem* describes when to apply the pattern. It explains the problem and its context [...] The *solution* describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template [...] The *consequences* are the results and trade-offs of applying the pattern... [7]

In this book, I present a subset of the original patterns by integrating them in the flow of the material when they become relevant. I do not reproduce the structured description that can be found in other pattern catalogs. I instead use a lightweight description for a pattern that focuses on the link between the problem and solution, and I include a discussion of important design decisions related to the pattern. I also prefer to refer to the problem as the *context* for applying a pattern, because design problems can sometimes be difficult to isolate. Finally, I will sometimes express the solution embodied by a pattern as a UML diagram that captures the name of the *abstract elements* of the pattern. Because these elements are abstract, I prefer to refer to them as a *solution template* rather than a solution. A typical task when attempting to apply a design pattern in a context is to map the abstract elements of the solution template to concrete design elements in the code. In the text, the name of design patterns are set in SMALL CAPS FONT. This is to indicate that a term refers to a well-defined design concept, as opposed to a general use of the term. For example, one design pattern is called the *Strategy* pattern. Instead of continuously referring to it as the *Strategy design pattern*, I will refer to it as the STRATEGY, which will distinguish it from the concept of a *strategy* as a general problem-solving approach.

Because solution templates for design patterns can be looked up in any number of resources, the most important skill to develop with respect to design patterns is to know when to apply them. For this reason, my coverage of design patterns emphasizes the rationale for using a pattern and a discussion of its strengths and weaknesses in different contexts, and de-emphasizes the focus on solution templates. One

potential pitfall when first learning about design patterns is to get over-enthusiastic and try to apply them everywhere. Like all other elements of design solutions, a particular instance of a design pattern will occupy a specific point in the design space, with attendant benefits and drawbacks. If I can make one generalization about the use of design patterns, it is that employing one tends to make an overall design more extensible. Sometimes, this extensibility is exactly what we need. At other times, it leads to unnecessary structures that clutter the code. In other words, using a particular design pattern in a particular way in a given context is a design decision which, like most other design decisions, should be critically assessed.

Design Antipatterns

An interesting take on the idea of a design pattern is that of a design *antipattern*. Just as it can be observed that some design solution elements recur between applications, it is also the case that recognizable flaws can be abstracted from many similar cases and catalogued. This influential idea took hold around the turn of the millennium in a popular book on refactoring, which documents 22 antipatterns as motivation to refactor the corresponding code [6]. Typical antipatterns include problems such as DUPLICATED CODE†, LONG METHOD†, and others that will be covered in this book. For reasons similar to design patterns, antipatterns are set in SMALL CAPS†, but are followed by a dagger symbol to distinguish them from actual patterns. Design antipatterns are also known as *code smells*, or *bad smells* (in code), to convey the idea of a symptom that something is not quite right.

Insights

This chapter introduced software design and placed it in the general context of software development projects.

- The verb *to design* refers to the process we follow when we design software, and the noun *a design* refers to the outcome of this process;
- The process of software design is the construction of abstractions of data and computation and the organization of these abstractions into a software solution;
- There is rarely a single solution to a design problem, only solutions that are better or worse in some dimensions;
- A design artifact is an external representation of one or more design decisions;
- Design is only one of many activities that take place during the development of a software system. Software development follows a process that can vary from organization to organization, and vary from planning-heavy to agile. Development processes typically involve iterations;
- A software development practice is a well-understood way of doing something to achieve a certain benefit. Examples include version control, coding conventions, and refactoring;

- Design knowledge can be captured in source code, code comments, specialized documents and diagrams, discussion forums, and models;
- The Unified Modeling Language, or UML, is a modeling language organized in terms of different types of diagrams. Using UML can be an effective way to illustrate different aspects of software without getting caught up in details;
- A design principle is a general idea that guides decision making when searching for a solution;
- A design pattern captures an abstract design solution that is applicable in a common design context. The description of a design pattern includes a name, a description of the design problem or context it addresses, a solution template, and a discussion of the consequences of applying the pattern;
- A design antipattern is an abstract description of a common design flaw.

Further Reading

The paper *Software Aging* by David L. Parnas [13] introduces the term *ignorant surgery* and provides a compelling motivation for the benefits of maintaining good design in software. Parnas is one of the early contributors to the software engineering discipline. Chapter 1 of the book *Clean Code: A Handbook of Agile Software Craftmanship* by Robert C. Martin [8] discusses the various ills of bad or “messy” code. My short paper titled *Sustainable Software Design* discusses in more detail what it means for design decisions to be self-evident [14].

Chapter 1 of the book *UML Distilled, 3rd Edition* by Martin Fowler [6] provides a more comprehensive introduction to the UML. Fowler distinguishes between three modes for using the UML: as sketches for design, as a blueprint for creating an application, and as source code that can be executed. Sketching is the mode employed in this book.

Among the many books that discuss software design principles, *Software Architecture: A Comprehensive Framework and Guide for Practitioners* by Oliver Vogel, Ingo Arnold, Arif Chughtai, and Timo Kehrer [17] is noteworthy for its organization of the different principles and of the relations between them.

The original book on design patterns is *Design Patterns: Elements of Reusable Object-Oriented Software* by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides [7]. This book is often referred to as the *Gang of Four* book. Because it predates the UML, the notation it uses for capturing software designs may feel a bit foreign. Nevertheless, it is a timeless reference work.

The book *Refactoring: Improving the Design of Existing Code*, also by Martin Fowler [4], is the main reference on the practice of refactoring. It introduces the idea of design antipatterns (which are called *code smells* in the book). Robert C. Martin also includes a list of bad smells in Chapter 17 of *Clean Code*, cited above.